

Laboratory report of Nanjing University of Information Science and Technology

Experiment Title Distributed Object Storage & Fault Tolerance Testing
Data 2026-6-25 Class Name Tang Yuanzhi ID 202383910019

一、Experimental purpose

Master Object Storage: Understand the difference between traditional file paths and S3-compatible Bucket/Object structures.

Experience Fault Tolerance: Observe how data remains accessible even when physical "disks" are destroyed.

二、Experiment Contents

In this experiment, students will simulate a distributed cloud object storage system similar to: Amazon S3, Alibaba Cloud OSS

using: MinIO, Docker Compose

Students will:

- Deploy a 4-disk distributed MinIO cluster

- Upload and manage cloud objects

- Observe backend storage fragmentation

- Simulate storage node failure

- Verify distributed storage fault tolerance

三、Detailed steps of the experiment

Step 1: Deploy a 4-Node Distributed Cluster

In your Codespaces terminal, create a dedicated directory for this lab:

```
New-Item -ItemType Directory -Name minio-lab -Force
Set-Location minio-lab
```

```
PS C:\Users\Lenovo> New-Item -ItemType Directory -Name minio-lab -Force

目录: C:\Users\Lenovo

Mode                LastWriteTime         Length Name
----                -
d-----          2026/6/25     9:00             minio-lab

PS C:\Users\Lenovo> Set-Location minio-lab
```

Step 2: Create Storage Directories

Create four independent storage directories to simulate four storage disks.

```
foreach ($dir in @("data1", "data2", "data3", "data4")) { New-Item -ItemType
Directory -Name $dir -Force }
```

```
PS C:\Users\Lenovo\minio-lab> foreach ($dir in @("data1", "data2", "data3", "data4")) {
>>     New-Item -ItemType Directory -Name $dir -Force
>> }
```

```
目录: C:\Users\Lenovo\minio-lab

Mode                LastWriteTime         Length Name
----                -
d-----          2026/6/25     9:01         data1
d-----          2026/6/25     9:01         data2
d-----          2026/6/25     9:01         data3
d-----          2026/6/25     9:01         data4
```

Step 3: Create docker-compose.yml

Create a file named docker-compose.yml in minio-lab with the following content:

```

PS C:\Users\Lenovo\minio-lab> @'
>> services:
>>   minio:
>>     image: quay.io/minio/minio
>>     container_name: cloud-minio
>>     volumes:
>>       - ./data1:/data1
>>       - ./data2:/data2
>>       - ./data3:/data3
>>       - ./data4:/data4
>>     ports:
>>       - "9000:9000"
>>       - "9001:9001"
>>     environment:
>>       MINIO_ROOT_USER: admin
>>       MINIO_ROOT_PASSWORD: password123
>>     command: server /data{1...4} --console-address ":9001"
>>
>>   mc:
>>     image: minio/mc
>>     container_name: cloud-mc
>>     depends_on:
>>       - minio
>>     entrypoint: /bin/sh
>>     tty: true
>> '@ | Set-Content docker-compose.yml -Encoding UTF8

```

Step 4: Explain the docker-compose.yml File

The meaning of each component in the docker-compose.yml file:

Component	Meaning
image	Docker image used to create the container
container_name	Custom name assigned to the container
volumes	Mount local directories into the container (data mapping)
ports	Port mappings between host and container
environment	Environment variables (MinIO credentials: admin/password123)
command	Start command: launch MinIO distributed storage on 4 drives with console at :9001
depends_on	Dependency: mc container waits for minio container to start

Step 5: Launch the Distributed Cluster

Start all services and verify running containers:

```

docker compose up -d
docker ps

```

```

PS C:\Users\Lenovo\minio-lab> docker compose up -d
unable to get image 'quay.io/minio/minio': failed to connect to the docker API at npipe:////./pipe/dockerDesktopLinuxEng
ine; check if the path is correct and if the daemon is running: open //./pipe/dockerDesktopLinuxEngine: The system canno
t find the file specified.
PS C:\Users\Lenovo\minio-lab> docker compose up -d
[+] up 19/19
✔Image minio/mc Pulled 44.4s
✔Image quay.io/minio/minio Pulled 13.9s
✔Network minio-lab_default Created 0.1s
✔Container cloud-minio Started 0.7s
✔Container cloud-mc Started 0.7s
PS C:\Users\Lenovo\minio-lab> docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS
d2b8fcb8fc42   minio/mc      "/bin/sh"               10 seconds ago Up 6 seconds
099e37d7db8f   quay.io/minio/minio "/usr/bin/docker-ent..." 10 seconds ago Up 7 seconds 0.0.0.0:9000-9001->9000-9001/tcp, [::]:9000-9001->9000-9001/tcp

```

Step 6: Configure MinIO Client Alias

Configure the MinIO client (mc) to connect to the MinIO server:

```
docker exec cloud-mc mc alias set myminio http://minio:9000 admin password123
```

Observation: Added myminio successfully.

Step 7: Create a Bucket

Create a cloud storage bucket:

```
docker exec cloud-mc mc mb myminio/my-cloud-storage
```

```

PS C:\Users\Lenovo\minio-lab> docker exec cloud-mc mc alias set myminio http://minio:9000 admin password123
Added 'myminio' successfully.
PS C:\Users\Lenovo\minio-lab> docker exec cloud-mc mc mb myminio/my-cloud-storage
Bucket created successfully 'myminio/my-cloud-storage'.

```

Step 8: Create and Upload a Large Test File

In distributed object storage systems, large files are automatically divided into multiple fragments and distributed across different storage disks. This demonstrates Erasure Coding and distributed storage mechanism.

8.1 Generate a 20MB test file:

```
fsutil file createnew bigfile.bin 20971520
```

8.2 Verify the file:

```
Get-Item bigfile.bin | Select-Object Name, Length
```

```

PS C:\Users\Lenovo\minio-lab> fsutil file createnew bigfile.bin 20971520
已创建文件 C:\Users\Lenovo\minio-lab\bigfile.bin
PS C:\Users\Lenovo\minio-lab> Get-Item bigfile.bin | Select-Object Name, Length

Name          Length
----          -
bigfile.bin 20971520

```

8.3 Copy file into mc container and upload:

```
docker cp bigfile.bin cloud-mc:/bigfile.bin
```

```
docker exec cloud-mc mc cp /bigfile.bin myminio/my-cloud-storage/
```

```
PS C:\Users\Lenovo\minio-lab> docker cp bigfile.bin cloud-mc:/bigfile.bin
Successfully copied 21MB to cloud-mc:/bigfile.bin
PS C:\Users\Lenovo\minio-lab> ^C
PS C:\Users\Lenovo\minio-lab> docker exec cloud-mc mc cp /bigfile.bin myminio/my-cloud-storage/
\bigfile.bin' -> 'myminio/my-cloud-storage/bigfile.bin'



| Total     | Transferred | Duration | Speed       |
|-----------|-------------|----------|-------------|
| 20.00 MiB | 20.00 MiB   | 00m00s   | 32.18 MiB/s |



PS C:\Users\Lenovo\minio-lab> docker exec cloud-mc mc ls myminio/my-cloud-storage/
[2026-06-25 01:26:04 UTC] 20MiB STANDARD bigfile.bin
```

Step 9: Verify Object Upload

List all objects in the bucket. The result shows bigfile.bin exists (see Figure 7 above).

Step 10: Observe Backend Storage Fragmentation

Observe how MinIO distributes data across multiple storage disks by listing contents of each data directory:

```
Get-ChildItem data1 -Recurse # (and data2, data3, data4)
```

```
Mode                LastWriteTime         Length Name
----                -
-a-----          2026/6/25      9:05             466 xl.meta

    目录: C:\Users\Lenovo\minio-lab\data4\.minio.sys\buckets\.bloomcycle.bin

Mode                LastWriteTime         Length Name
----                -
-a-----          2026/6/25      9:27             591 xl.meta

    目录: C:\Users\Lenovo\minio-lab\data4\.minio.sys\buckets\.usage-cache.bin

Mode                LastWriteTime         Length Name
----                -
-a-----          2026/6/25      9:27             533 xl.meta

    目录: C:\Users\Lenovo\minio-lab\data4\.minio.sys\buckets\.usage-cache.bin.bkp

Mode                LastWriteTime         Length Name
----                -
-a-----          2026/6/25      9:27             533 xl.meta

    目录: C:\Users\Lenovo\minio-lab\data4\.minio.sys\buckets\.usage.json

Mode                LastWriteTime         Length Name
----                -
-a-----          2026/6/25      9:27             973 xl.meta
```

Observations:

- Object metadata stored as xl.meta files

- Fragmented storage structure with multiple part directories
- Distributed object placement across all 4 storage nodes

Cloud Object Storage Architecture

Object ↓ Metadata (xl.meta) ↓ Chunk Splitting ↓ Distributed Parts ↓ Multiple Storage Nodes

Step 11: Simulate Disk Failure

Distributed cloud storage systems must tolerate hardware failures.

11.1 Stop the cluster:

```
docker compose down
```

11.2 Delete one storage disk (simulate disk failure):

```
Remove-Item data3 -Recurse -Force
```

```
New-Item -ItemType Directory -Name data3 -Force
```

```
PS C:\Users\Lenovo\minio-lab> Remove-Item data3 -Recurse -Force
PS C:\Users\Lenovo\minio-lab> New-Item -ItemType Directory -Name data3 -Force

目录: C:\Users\Lenovo\minio-lab

Mode                LastWriteTime         Length Name
----                -
d-----          2026/6/25          9:28         data3
```

11.3 Restart the cluster:

```
docker compose up -d
```

Step 12: Verify Fault Tolerance

Check whether objects are still accessible:

```
docker exec cloud-mc mc ls myminio/my-cloud-storage/
```

```
PS C:\Users\Lenovo\minio-lab> docker compose down
[+] down 3/3
✓Container cloud-mc      Removed
✓Container cloud-minio   Removed
✓Network minio-lab_default Removed
PS C:\Users\Lenovo\minio-lab> Remove-Item data3 -Recurse -Force
PS C:\Users\Lenovo\minio-lab> New-Item -ItemType Directory -Name data3 -Force

目录: C:\Users\Lenovo\minio-lab

Mode                LastWriteTime         Length Name
----                -
d-----          2026/6/25          9:28         data3

PS C:\Users\Lenovo\minio-lab> docker compose up -d
[+] up 3/3
✓Network minio-lab_default Created
✓Container cloud-minio   Started
✓Container cloud-mc      Started
PS C:\Users\Lenovo\minio-lab> docker exec cloud-mc mc ls myminio/my-cloud-storage/
mc: <ERROR> Unable to list folder. Requested path `/myminio/my-cloud-storage` not found
```

Observation Result: After deleting data3 and restarting, the bucket myminio/my-cloud-storage could not be found. This indicates that in a 4-node MinIO deployment, when an entire storage node's data is permanently lost (not just temporarily offline), the bucket metadata and/or enough erasure coding parity blocks were destroyed beyond recovery threshold.

Step 13: Use git to save your changes

Save the experiment work to a git repository for version control.

四、Reflection

1. Problems Encountered and Proposed Solutions

(1) PowerShell syntax compatibility issue: The original experiment instructions used Bash commands (e.g., heredoc syntax `<<'EOF'`), which do not work in Windows PowerShell. Solution: Converted all Bash commands to their PowerShell equivalents (e.g., `New-Item` instead of `mkdir -p`, `Set-Content` with here-string `@'...'@` instead of `cat > file << EOF`).

(2) Creating multiple directories at once: PowerShell's `New-Item` does not support creating multiple directories in a single `-Name` parameter. Solution: Used a `foreach` loop to iterate over directory names.

(3) Generating large random files: PowerShell does not recognize '20MB' as a valid numeric literal. Solution: Used the exact byte count 20971520 (20 x 1024 x 1024) or the built-in Windows utility `fsutil file createnew`.

(4) Multi-line code pasting: When pasting multi-line commands line-by-line into PowerShell, complex expressions with parentheses may get truncated during copy-paste. Solution: Used simpler single-line alternatives like `fsutil` instead of `.NET` multi-line expressions.

2. Why was data unavailable after deleting one storage directory?

In this experiment, after permanently removing the data3 directory and restarting the cluster, the uploaded object became inaccessible. There are several reasons for this. First, MinIO uses Erasure Coding (EC) to split data into data blocks and parity blocks distributed across all nodes; in a default 4-node configuration, the EC scheme may use approximately $N=2$ data blocks plus $M=2$ parity blocks, which theoretically allows tolerating the loss of 1 node. However, when a node's data is completely deleted rather than just taken offline, the critical bucket metadata (stored as `.minio.sys/buckets` metadata) may reside on that node and cannot be reconstructed from other nodes. Second, MinIO requires a quorum of nodes to maintain consistency; with 4 nodes where 1 has been wiped clean, if the `format.json` or other initialization metadata on that node is missing, the cluster may not form properly. Third, the fault tolerance mechanism primarily handles temporary node failures (disk offline, network partition) rather than permanent data loss scenarios. In production environments, more nodes (typically 8 or 16) and higher EC redundancy levels (e.g., EC:4) are used to ensure data survives multiple simultaneous node failures.

